Olozzo.com: Stripe Production Runbook

The execution sequence for an **approved** production deployment. Full command details (purpose, location, permissions, expected result, risk) are in the Live Deployment Guide §7, and the step IDs here (C-…) refer to it. Sign-offs are in the Launch Checklist.

> **No command in this runbook has been run against production.** Commands marked 🟡 or 🔴 must be run by a person after the named approval gate. Hosting is unconfirmed (Deployment Guide §12): use only the variant that matches the confirmed host, **[Vercel]** or **[Node host]**.

Classes: 🟢 read-only · 🟡 changes configuration or the repository · 🔴 changes production (approval + maintenance window)

Roles: **DEV** developer · **OPS** server admin / DevOps · **CLI** client (Stripe account owner) · **PM** project manager

Approval gates

Gate	Condition to pass	Approved by	Evidence
G0 Release ready	Blockers B1 (code committed and merged) and B6 (no demo accounts) closed; C-A1–C-A11 pass	DEV lead	Tag `<RELEASE_TAG>`, checks output
G1 Stripe account ready	Account owned by the client, verified, payouts configured, developer invited (B4)	CLI	Launch Checklist part A
G2 Hosting known	Deployment Guide §12 items supplied (B3)	OPS	Filled-in §12
G3 Live credentials	Client authorises creation and storage of live key and webhook	CLI	Launch Checklist part D
G4 Deployment window	Date and time agreed (low traffic), rollback owner named, B5 (legal pages) closed	PM + CLI	Calendar entry
G5 Real-payment test	Amount, card owner, refund owner agreed	CLI	Launch Checklist part D
G6 Launch sign-off	§6 and §7 passed	CLI + PM	Launch Checklist part E

1. Pre-deployment checks (G0)

Step	Owner	Action	Class	Done when
1.1	DEV	C-A1 branch and working tree	🟢	Status empty on the release commit
1.2	DEV	C-A2 review the release commit	🟢	Reviewed PR merged
1.3	DEV	C-A3 Node ≥ 20 (local and build host)	🟢	`v20+`
1.4	DEV	C-A4–C-A8 install, type check, tests, build	🟢	All pass

Step	Owner	Action	Class	Done when
1.5	DEV	C-A9 secret scan of `.next/static`	🟢	No output
1.6	DEV	C-A10 no env files in Git	🟢	No output
1.7	DEV	C-A11 deployment configuration review	🟢	Matches the confirmed host
1.8	DEV	C-A12 tag `<RELEASE_TAG>`	🟡	Tag pushed
1.9	DEV / OPS	Production demo-account check (B6)	🟢	No `@olozzo.com` demo accounts with the default password, or they are removed or reset

2. Required approvals

G1 and G2 must pass before §3; G3 before §3.3 and §4; G4 before §3.4 and §5; G5 before §7; G6 closes the launch.

3. Environment configuration

3.1 Variables (production)

`STRIPE_SECRET_KEY` = `<LIVE_KEY>` (`rk_live_…` preferred) · `STRIPE_WEBHOOK_SECRET` = `<LIVE_WEBHOOK_SIGNING_SECRET>` · `NEXT_PUBLIC_APP_URL` = `https://<PRODUCTION_DOMAIN>` (set **before** the build) · `ENABLE_DEMO_PAYMENTS` unset or `false`. No publishable key. Details: Deployment Guide §4.

3.2 Secret-handling rules

Never put a secret on a command line, in Git, chat, email, tickets or logs; never print environment values to "check" them. Use the host's secret store; read local secrets with `read -rs NAME && export NAME`, then `unset NAME`.

3.3 Store the live credentials (after §4 produces them)

Step	Owner	Action	Class
3.3.1	DEV / OPS	[Vercel] C-B1 list names; C-B2 add the three variables (Production scope only, Sensitive)	🟡
3.3.2	OPS	[Node host] C-B3 edit `<ENV_FILE_PATH>`, `chmod 600`	🟡
3.3.3	DEV	Confirm no live Stripe variables in Preview / Development / staging scopes	🟢

3.4 Database schema (G4, maintenance window)

Step	Owner	Action	Class	Done when
3.4.1	OPS	C-C1 backup (Neon branch or `pg_dump`)	🟡	Backup verified restorable
3.4.2	DEV	C-C2 preview the diff	🟢	Only `ADD COLUMN`, `CREATE INDEX`, `CREATE TABLE "StripeEvent"`; **no** `DROP`
3.4.3	DEV + OPS	C-C3 `npx prisma db push`	🔴	"in sync"; abort on any data-loss warning

Never on production: `npm run setup`, `db:seed`, `db:reset`, `--force-reset`, `--accept-data-loss`. (`db:seed-catalog` is designed to be additive and safe on the live database, per its header, but it isn't part of this deployment.)

4. Live API key and webhook (G3)

4.1 Live API key

Step	Owner	Action
4.1.1	CLI	Invite DEV to the Stripe account as a team member (no shared login)
4.1.2	DEV	Live mode → Developers → API keys → create a restricted key `olozzo-production`: Checkout Sessions **Write**, Refunds **Write**, PaymentIntents **Read** (permissions confirmed in staging first)
4.1.3	DEV	Copy it **once**, straight into the secret store (§3.3); optionally attach an access policy

4.2 Live webhook endpoint

Step	Owner	Action
4.2.1	DEV	Live mode → Workbench → Webhooks → Create event destination → Your account → **Snapshot**, API version `2026-09-30.endive`
4.2.2	DEV	Events: `checkout.session.completed`, `checkout.session.async_payment_succeeded`, `checkout.session.async_payment_failed`, `checkout.session.expired`, `charge.refunded`, `charge.dispute.created`
4.2.3	DEV	URL `https://<PRODUCTION_DOMAIN>/api/webhooks/stripe` (final URL, no redirect)
4.2.4	DEV / OPS	Signing secret → `STRIPE_WEBHOOK_SECRET` (§3.3)

4.3 Live account settings (CLI + DEV)

Payment methods (live), statement descriptor, email receipts, Adaptive Pricing, Radar, dispute notifications.

5. Deployment (G4)

Step	Owner	Action	Class
5.1	PM	Announce the window; name the rollback owner	none
5.2	DEV / OPS	Confirm §3.4 is done (schema first)	🟢
5.3	DEV	[Vercel] C-C4 `vercel deploy --prod` from `<RELEASE_TAG>` (or merge to the production branch)	🔴
5.4	OPS	[Node host] C-C5 checkout tag → `npm ci` → `npm run build` → `<RESTART_COMMAND>`	🔴

Live payments become possible as soon as this deployment serves traffic with the live key; there is no separate switch. Go straight to §6.

6. Post-deployment verification

Step	Owner	Command (Deployment Guide)	Expected
6.1	DEV	C-D1 HTTPS and certificate	`200`; valid certificate
6.2	DEV / OPS	C-D2 service status	Ready / running
6.3	DEV	C-D4 webhook probe	`400`, no redirect
6.4	DEV	C-D5 `/admin/payments` (and C-D5b on a Node host)	**live mode**, webhook secret set, demo payments off, https URL
6.5	DEV	C-D6 open Stripe checkout, don't pay	EUR total, no "Sandbox" badge
6.6	DEV	C-D3 + C-D7 logs	No Stripe or Prisma errors; no secrets
6.7	OPS	C-D8 egress to Stripe ([Node host])	`401`

Any failure in 6.1–6.6 → fix within the window or roll back (§10).

7. Controlled real-payment test (G5)

1. Agree in writing: product (cheapest total seen about €8.44 incl. shipping), card owner (client side), refund owner, and whether the supplier is warned.
2. Buy on `https://<PRODUCTION_DOMAIN>`.
3. Verify (C-D9): Stripe live Payments shows it; Event deliveries `checkout.session.completed` `200`; order **Paid / Processing**; supplier order(s) created.
4. Cancel the supplier order or warn the supplier so nothing ships.
5. Refund from admin order → **Refund €...**. Verify order **Refunded**, supplier orders **Cancelled**, `charge.refunded` `200`.
6. Record date, order number, amount, people involved and the result in the Launch Checklist. Stripe's processing fee may not be returned.

8. Monitoring (first two weeks)

What	Where	Owner	Frequency
Webhook failures	Stripe → Webhooks → Event deliveries	DEV	Daily
Needs review, long-pending, unpaid > 2 h	`/admin/payments`	DEV	Daily
Disputes	Stripe → Disputes; email notifications	CLI	Daily
Server errors	C-D3 log search	DEV / OPS	Daily
Balances and payouts	Stripe → Balances / Payouts	CLI	Weekly

9. Incident response

Incident	First action	Owner
Checkout fails for everyone	C-D5 + C-D3; key issue → restore the previous live key (§10.2); otherwise §10.1	DEV
Webhooks failing	Fix the secret or URL; Stripe retries 3 days; **Sync payment** on affected orders	DEV

Incident	First action	Owner
Sandbox key in production	Set the live key **and** live secret; redeploy; cancel supplier orders of `cs_test_` orders	DEV + PM
Suspected key leak	Rotate in Stripe; update the secret; redeploy; review Workbench logs	CLI + DEV
Unresolved payment discrepancy	Reconcile (§10.1 step 4); escalate PM → CLI → Stripe support	PM

Full table: Deployment Guide §13.

10. Rollback

10.1 Code rollback (🔴, PM approves)

1. Record Unpaid / Pending orders from the last 60 minutes and the live payments of the last hour.
2. [Vercel] `vercel rollback <PREVIOUS_DEPLOYMENT_URL>` (or Instant Rollback) · [Node host] `git checkout <PREVIOUS_RELEASE_TAG> && npm ci && npm run build && <RESTART_COMMAND>`
3. Keep the database (additive change). Restore the backup only for data corruption, with explicit approval: it loses newer orders.
4. Reconcile each recorded payment: Stripe paid but order unpaid → **Sync payment**; double charge → refund the duplicate. Don't ask customers to pay again until Sync payment confirms the order is unpaid.
5. Re-run §6.1–6.4; check webhook deliveries.

10.2 Configuration rollback (🟡)

Restore the previous **live** key and webhook secret as a pair from the secret store's history; redeploy or restart; re-run §6.4. **Never fall back to sandbox keys** on production.

11. Information needed (from OPS)

Deployment Guide §12: hosting and deploy method, domain and HTTPS, secret storage, Node host paths and commands, DB backup method, current deployment, staging, firewall rules.

12. Commands requiring special caution

Command	Why
`npx prisma db push` (production)	Alters the live schema
`npm run setup` / `db:seed` / `db:reset`	Wipe the database and create demo accounts with a published password. **Never on production**
`npm run db:seed-catalog`	Additive catalog import (safe on live per `prisma/seed-catalog.ts`); run `-- --dry-run` first; not part of the Stripe deployment
`vercel deploy --prod`, `<RESTART_COMMAND>`	Customer-facing change; activates live payments
`vercel env add/rm … production`, editing `<ENV_FILE_PATH>`	Changes or breaks the payment mode

Command	Why
`vercel rollback`, checking out an older tag	Reverts features; reconciliation needed
Rotating or expiring Stripe keys	Payments fail until the new key is deployed
`pg_dump`	Output contains personal data
`stripe listen` / `stripe trigger`	Sandbox/development only; never against production